AI-Native Engineer Capstone

WealthWise — Robo-Advisory & Portfolio Recommendation Platform

Business Case BC-AINE-008 · Domain: BFS — Wealth Management

1. Project Identity

Business Case Title	Robo-Advisory & Portfolio Recommendation Platform
Business Case ID	BC-AINE-008
Domain	BFS — Wealth Management
Project Type	AI-Native Engineering Capstone
Cohort	(filled by Operations team)

2. Engagement Overview

Duration	20 hours
Format	Individual
Evaluation Mode	Automated review of submitted Git repository against the AI-Native Engineering Rubric (21 parameters / 100 marks across 5 categories)
Submission	Push final code to your assigned Virtusa GitLab repository by the cohort cut-off
Review Output	Per-learner Excel report (Summary, Categories, Scorecard, Detailed, Improvement)
Grade Bands	Distinction ≥ 90 · Merit 75–89 · Pass 60–74 · Not Yet Passed < 60

What is evaluated: AI-native engineering practices (specs, agents, skills, hooks, CLAUDE.md hierarchy, plugin packaging, MCP, Harness plugin usage), business understanding (context, rules, AC), architectural discipline (structural rules, TDD), technical implementation (code, tests, coverage, UI validation), and CI/CD integration (PR-driven workflow, Claude in CI).

What is not evaluated: the specific technology stack chosen, the visual polish of the UI, or the use of any AI provider beyond Claude Code.

3. Problem Statement & Expected Solution

3.1 Problem

A wealth management firm wants to launch self-service robo-advisory for retail investors. You build customer risk profiling, rule-driven portfolio recommendation, goal tracking, and rebalancing nudges —

under one constraint: no manual coding. Recommendation logic is deterministic and rule-based (not ML). Every line of production code must be generated by Claude Code agents under your supervision.

3.2 Your Role

AI-Native Engineer — architect, supervisor, and quality gatekeeper. You write specifications, configure agents, install guardrails, and review pull requests. You do not write production code by hand.

3.3 Expected Solution

A working platform delivered as a Git repository that demonstrates:

- A complete AI-native engineering substrate built on the Claude Harness Engine plugin foundation, extended with wealth-advisory-specific specs, agents, skills, hooks, commands, and MCP integration.
- A functional robo-advisory platform built end-to-end by Claude Code agents — covering risk profiling, allocation recommendation, drift detection, and rebalancing — with no hand-written production code.
- An evidence trail (PRs, agent logs, post-mortems, sprint contracts) proving the work was agent-led.

3.4 Applicable Rules

- **No-Hand-Coding Rule.** All production code, tests, and migrations must be generated by Claude Code agents. Hand-edits are limited to specs, CLAUDE.md, agent definitions, hooks, skills, and other substrate files.
- **Spec-Is-Truth Rule.** Whenever a spec and code disagree, the spec wins. Update the spec deliberately and regenerate the code.
- **PR-Only Merge Rule.** No direct commits to main. All changes flow through agent-reviewed pull requests.
- **Synthetic-Data Rule.** Use only dummy data. Do not use Virtusa confidential or customer-sensitive data for this project.

4. Technology Stack

Choose any stack that matches your background. The stack does not affect evaluation.

Layer	Options
Frontend	React · Angular · Vue · server-rendered (Thymeleaf / Razor)
Backend	Java + Spring Boot · .NET + ASP.NET Core Web API · Python + FastAPI · Node.js + Express / NestJS
Database	In-memory (H2 / SQLite) · RDBMS (MySQL / MariaDB / SQL Server) · NoSQL (MongoDB / DynamoDB)
Build & Dependency Management	Maven / Gradle · dotnet CLI · pip + Poetry · npm / pnpm
Testing	JUnit / NUnit / pytest / Vitest · Playwright (E2E) · ArchUnit / dependency-cruiser / import-linter

Layer	Options
CI	GitLab CI (preferred for Virtusa) · GitHub Actions, with Claude Code Action integrated
MCP Server	Playwright MCP
Required Plugin	Claude Harness Engine plugin (Generator–Evaluator harness for autonomous long-running development)

5. Acceptance Criteria & Non-Functional Requirements

5.1 Functional Acceptance Criteria

ID	Criterion
AC-01	Customer can complete a risk-profile questionnaire of at least 6 questions; a deterministic scoring rule assigns a risk band (CONSERVATIVE / MODERATE / AGGRESSIVE)
AC-02	Risk band determines a recommended asset allocation per a versioned template; allocation percentages always sum to 100
AC-03	Customer can create financial goals with target_amount, target_date, and priority; multiple goals per customer are supported
AC-04	Portfolio-recommendation engine returns an allocation that maps to the customer's risk band and goal horizon; output is deterministic given the same inputs
AC-05	Customer holdings track current value per asset class; allocation drift = abs(current% − target%) is calculated for each asset class
AC-06	Rebalancing recommendation is generated when any asset class's drift exceeds the configured threshold (e.g., 5%)
AC-07	Rebalancing engine proposes BUY / SELL quantities with a recommendation ID; customer can accept or dismiss with audit
AC-08	Goal progress is reported as percent_complete = current_goal_value / target_amount; updated on each daily NAV refresh
AC-09	Advisor can override a customer's risk band with reason and supporting note; the override is audited with actor and timestamp
AC-10	Risk-band rules and asset-allocation templates are versioned; published versions are immutable; in-flight customers continue on the version active at the time of their recommendation

Note. *Each AC must have at least one test referencing its AC-NN identifier.*

5.2 Non-Functional Requirements

ID	Requirement
NFR-01	Portfolio values, allocation percentages, and drift are computed in fixed-point — never floating-point
NFR-02	Risk-band assignments, recommendations, rebalancing actions, and overrides are append-only
NFR-03	Customer PII and financial-position data are never written to logs
NFR-04	Authentication boundary at controller layer; customer / advisor / admin roles are separated
NFR-05	Database, policy, and template migrations are append-only
NFR-06	Structured JSON logs with request correlation IDs
NFR-07	Health endpoint returns 200 within 1 second of a successful startup
NFR-08	Architecture rules enforced as tests — allocation percentages must sum to 100; published templates cannot be mutated; risk-band reassignment requires an override audit record

6. Functional Scope

6.1 Customer / End-User Functionality (User Journeys)

- **Risk Profile.** Complete the risk-profile questionnaire; see assigned risk band.
- **Goals.** Set financial goals (retirement, education, home) with target amount and date.
- **Recommendation.** View recommended asset allocation per risk band and goal horizon.
- **Holdings.** View current holdings and allocation drift per asset class.
- **Rebalance.** Receive rebalancing recommendations when drift exceeds threshold; accept or dismiss.

6.2 Internal & System Functionality

- **Advisor Workbench.** Customer portfolio view, override risk band with reason, log manual recommendation.
- **Asset Price Feed (stub).** Daily NAV updates per asset class on a configurable schedule.
- **Rebalancing Engine.** Detect drift > threshold, propose BUY / SELL quantities, generate stubbed orders.
- **Goal Tracker.** Compute and snapshot progress per goal on each NAV refresh.
- **Admin.** Edit risk-band rules and questionnaire scoring; manage asset-class master and versioned allocation templates.

6.3 Out of Scope

You do not need to build:

- Real exchange or brokerage integration
- Real-time market data (stubbed daily NAV only)

- Multi-currency or FX
- Tax-loss harvesting and tax optimization
- Real KYC document workflow (use a stub flag)
- Machine-learning-based recommendation (rule-based only)
- Production deployment, secret management, multi-region

7. AI-Native Implementation Expectations

Foundation. Install the **Claude Harness Engine plugin** first. The harness ships 7 base agents (planner, generator, evaluator, security-reviewer, test-engineer, design-critic, ui-designer), 12+ hooks, base commands, sprint contracts, an evaluator wired to Playwright MCP, and a self-healing loop. On top of that foundation, your repository must add the project-specific substrate listed in this section. The rubric scores the final repository — both Harness-supplied and learner-added artifacts contribute, but the thresholds below are calibrated so that learners who only install Harness without customizing for the project domain will not meet them.

7.1 Claude Code Specs (40 marks)

- specs/app_spec.md (root spec) and specs/<feature>_spec.md per major feature, each with explicit Acceptance Criteria sections
- Layered CLAUDE.md — root + per-module + per-folder; AGENTS.md as a table of contents only
- At least 3 project-specific skills under .claude/skills/ added by you (examples: risk-band-scorer, allocation-template-resolver, drift-calculator, rebalancing-quantity-proposer, goal-progress-tracker; or technical: policy-version-validator, archtest-author, spec-to-test-generator)
- At least 2 project-specific custom commands under .claude/commands/ added by you (beyond Harness defaults)
- At least 2 project-specific hooks under .claude/hooks/ added by you (examples: allocation-sum-invariant-check.sh, template-immutability-check.sh, pii-redaction-check.sh)
- At least 2 project-specific agents under .claude/agents/ added by you. These may be **domain agents** (e.g., risk-profiler-agent, recommendation-engine-agent, rebalancing-agent, goal-tracker-agent), **technical agents** that strengthen the engineering substrate (e.g., policy-version-validator-agent, archtest-author-agent, simulation-runner-agent), or a mix of both. You may rename Harness's generator → implementer, evaluator → reviewer, test-engineer → tester to align with canonical naming
- At least one programmatic Claude Agent SDK usage in scripts/
- Your own plugin.json packaging your project's substrate (in addition to Harness's plugin.json)
- At least one project-specific debugging-log or post-mortem document under docs/ evidencing environment-first resolution
- Playwright MCP configured in .mcp.json and actively used by the evaluator

7.2 Business Understanding (20 marks)

- docs/business-case.md covering: problem, target users, success metrics, domain rules, value proposition. This is the document the reviewer plugin reads — see Section 8
- At least 4 rule / policy / validator files under src/domain/ (or equivalent) implementing wealth-advisory business rules

- Frontend evidence (any of: React / Angular / Vue) with at least one responsive layout
- Acceptance criteria expressed in testable form — Given-When-Then or AC-NN identifiers — for at least 4 specs

7.3 Architectural Discipline (8 marks)

- Architecture-test directory (tests/architecture/ or equivalent) with at least 3 structural tests
- ArchUnit / dependency-cruiser / import-linter configured to enforce layering rules
- TDD discipline document at docs/tdd.md and at least 10 test files committed across history showing red–green–refactor pattern

7.4 Technical Implementation (26 marks)

- docs/architecture.md with at least 1 architecture diagram (Mermaid or image) and an explicit layered structure (controllers / services / repositories / domain)
- Substantive codebase with at least 3000 lines of meaningful generated code
- Playwright (or equivalent) for UI validation with at least one snapshot directory
- Tests tagged with AC IDs — every spec AC has at least 1 corresponding test
- At least 20 unit tests with a coverage artefact (coverage.xml, lcov.info, or equivalent) and the coverage tool declared in dependencies

7.5 CI/CD (6 marks)

- .gitlab-ci.yml (or .github/workflows/) with the build and test pipeline
- Claude Code Action (or equivalent) wired into the CI workflow
- At least 3 PR-driven merges (use git merge --no-ff to preserve PR history); zero direct commits to main

8. Expected Outcomes & Deliverables

By the end of 20 hours, the submitted Git repository must contain the Mandatory items below. Good-to-Have items differentiate Merit and Distinction submissions.

8.1 Mandatory

1. **Working Application.** Robo-advisory platform deployable locally via a single command (./mvnw spring-boot:run, npm start, dotnet run, or equivalent) with seed data (asset classes, allocation templates, sample customers) and a README quick-start.
2. **docs/business-case.md.** The business context document the reviewer plugin reads. Use this brief's Sections 3 and 6 as the source; rewrite in your own words covering problem, users, success metrics, domain rules, and value proposition.
3. **Specifications.** specs/app_spec.md plus per-feature specs (risk-profile, recommendation, holdings, rebalancing, goal-tracker, advisor-workbench) with explicit Acceptance Criteria sections.
4. **Layered CLAUDE.md Hierarchy.** Root + per-module + per-folder; AGENTS.md as a table of contents.
5. **Project Agent Substrate.** .claude/agents/, .claude/skills/, .claude/commands/, .claude/hooks/ — Harness foundation plus your project-specific additions as listed in Section 7.1.

6. **Claude Harness Engine Plugin Integration.** Plugin installed; project scaffolded via the harness; at least one full sprint cycle (Generator → Evaluator → ratcheting → PR) driven through it. Sprint contracts and evaluator outputs committed under sprint-contracts/ and specs/reviews/.
7. **Plugin Manifest.** Your own plugin.json at the repository root packaging your wealth-advisory substrate; .mcp.json with Playwright MCP active.
8. **Test Suite.** Unit tests (incl. risk-band scoring rules, allocation invariants, drift calculations, rebalancing math), architecture tests, AC-tagged tests, Playwright UI tests, coverage report — all committed.
9. **CI Pipeline.** .gitlab-ci.yml (or equivalent) with Claude Code Action wired in; passing build on main.
10. **PR-Driven Git History.** At least 3 PR-driven merges; zero direct pushes to main.

8.2 Good-to-Have

11. **Architecture & TDD Documentation.** docs/architecture.md with at least one diagram (C4 context or sequence diagram for the risk-profile-to-recommendation flow); docs/tdd.md describing the red–green–refactor approach with one worked example (e.g., AC-02 allocation-sum invariant).
12. **Autonomous Fix Loop Evidence.** At least one recorded loop where an agent detected a failure (e.g., an allocation-sum invariant violation after a template version change), reproduced, fixed, validated, and opened a PR. Capture the trace under docs/fix-loops/.
13. **Knowledge Deposit Log.** docs/knowledge-deposits.md recording recurring mistakes encoded back into rules, hooks, or skills (e.g., a hook that rejects any allocation template that does not sum to exactly 100).
14. **Multiple Sprint Cycles.** More than one harness-driven sprint covering: risk-profile + recommendation → holdings + drift → rebalancing + goal-tracker.
15. **Bonus Agents.** Beyond the required additions: janitor, performance-auditor, doc-writer, or other domain-specific or technical agents.

On completion of the capstone, you will have demonstrated the skill the industry is seeking: building a rule-driven robo-advisory platform with risk profiling, portfolio recommendation, and rebalancing logic — without writing the code yourself, while maintaining full control of quality, security, and architecture through AI-native engineering practices.